

Digital System Design

LAB IV Counter

DONG Yunyang

dongyy@sustech.edu.cn

411, No. 2, Hui Yuan

Tencent Meeting: 874-068-9694

Review – 3-8 decoder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Decoder is
    Port ( en_n : in std_logic;
          a : in std_logic_vector(2 downto 0);
          y_n : out std_logic_vector(7 downto 0));
end Decoder;
```

```
set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports { a[0] }];
set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 } [get_ports { a[1] }];
set_property -dict { PACKAGE_PIN M13 IOSTANDARD LVCMOS33 } [get_ports { a[2] }];
set_property -dict { PACKAGE_PIN R15 IOSTANDARD LVCMOS33 } [get_ports { en_n }];

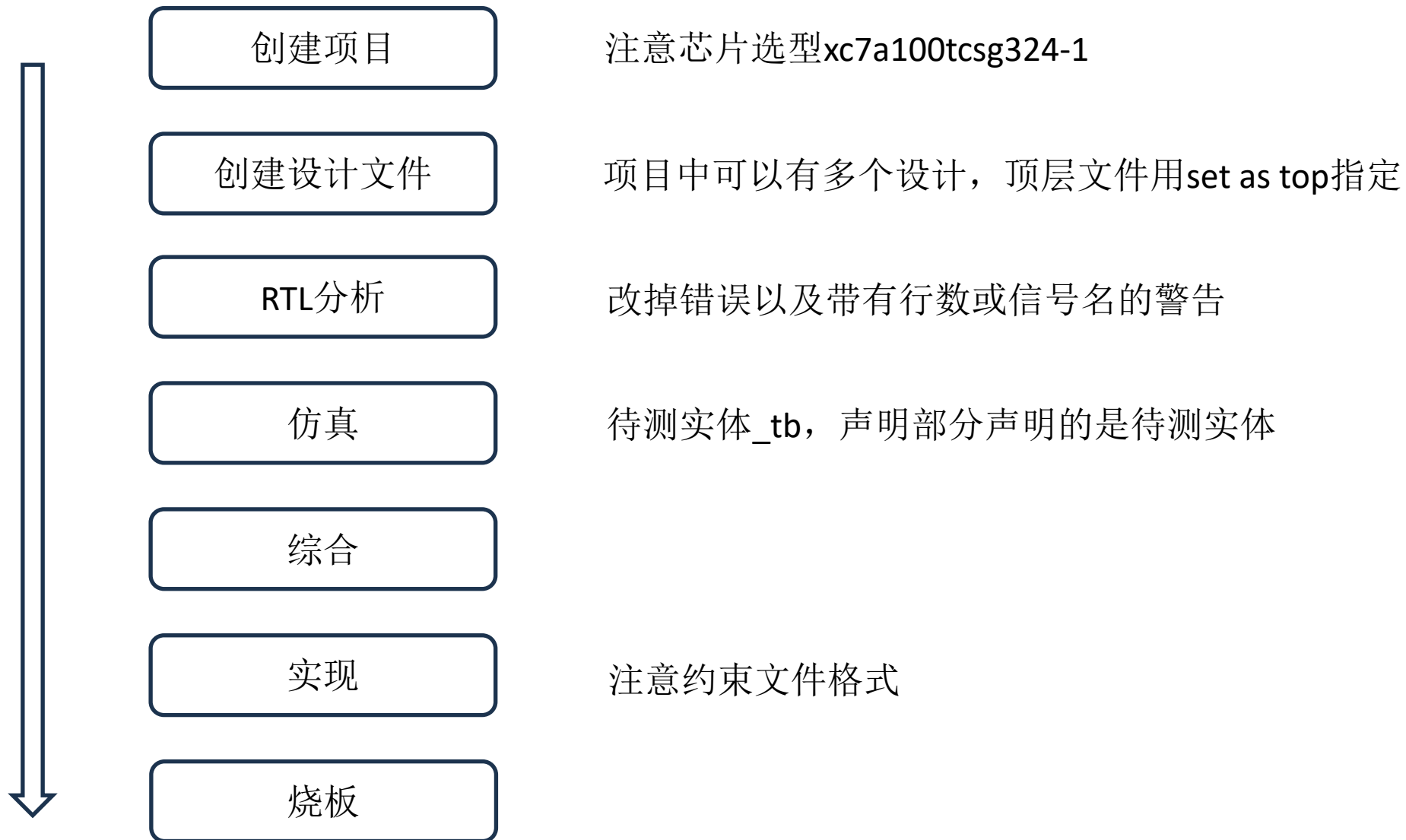
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports { y_n[0] }];
set_property -dict { PACKAGE_PIN K15 IOSTANDARD LVCMOS33 } [get_ports { y_n[1] }];
set_property -dict { PACKAGE_PIN J13 IOSTANDARD LVCMOS33 } [get_ports { y_n[2] }];
set_property -dict { PACKAGE_PIN N14 IOSTANDARD LVCMOS33 } [get_ports { y_n[3] }];
set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports { y_n[4] }];
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { y_n[5] }];
set_property -dict { PACKAGE_PIN U17 IOSTANDARD LVCMOS33 } [get_ports { y_n[6] }];
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { y_n[7] }];
```

architecture Behavioral of Decoder is
begin

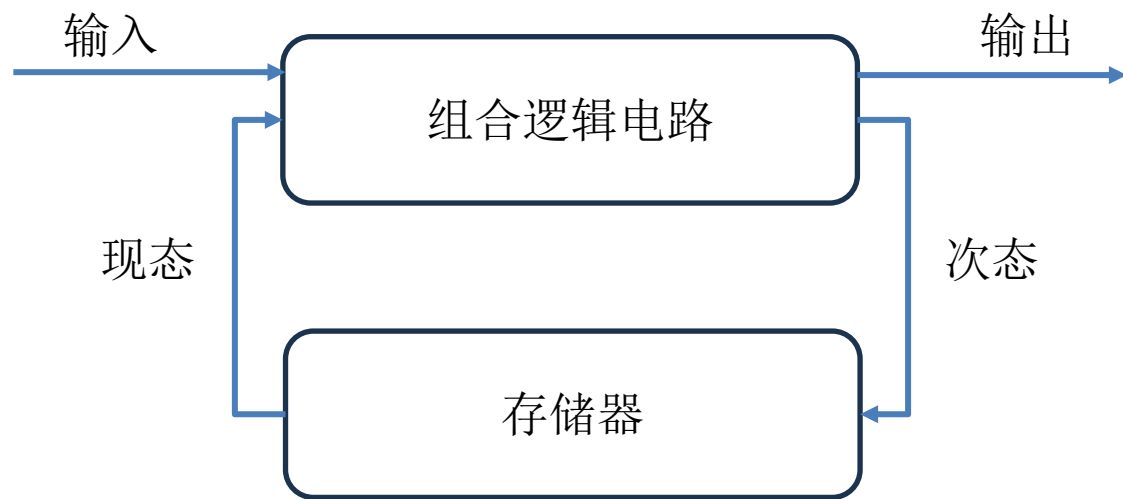
```
    process(a, en_n)
    begin
        if (en_n = '1') then
            y_n <= "11111111";
        else
            case a is
                when "000" => y_n <= "11111110";
                when "001" => y_n <= "111111101";
                when "010" => y_n <= "111111011";
                when "011" => y_n <= "111110111";
                when "100" => y_n <= "111011111";
                when "101" => y_n <= "110111111";
                when "110" => y_n <= "101111111";
                when "111" => y_n <= "011111111";
                when others => y_n <= "XXXXXXXX";
            end case;
        end if;
    end process;
end Behavioral;
```

y_n, en_n: n represents for 低电平有效

Review – Design Flow



Sequential circuit

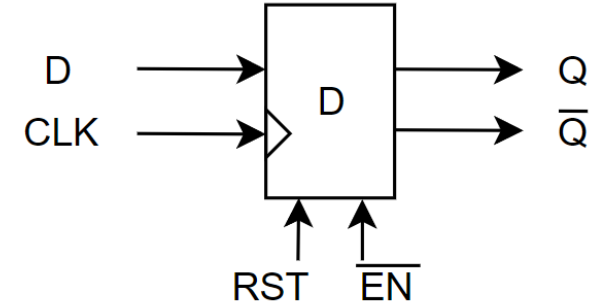


时序逻辑电路的输出不仅与当前的输入有关，而且与系统的状态有关。

相当于在组合逻辑的输入端加上了一个反馈输入，反馈输入由存储器记录了系统的状态。

因此，时序电路不仅仅由逻辑门组成，还包含有存储信息的有记忆能力的电路：触发器、寄存器等。

Example: D Flip-Flop



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DFF is
    Port ( clk : in std_logic;
           reset : in std_logic;
           enable : in std_logic;
           d : in std_logic;
           q : out std_logic;
           q_n : out std_logic);
end DFF;
```

```
architecture Behavioral of DFF is
    signal q_next : std_logic;
begin
```

```
    process(clk,reset)
```

只需要列出时钟和异步信号

```
    begin
```

```
        if (reset = '1') then
```

异步功能

```
            q_next <= '0';
```

```
        elsif (clk'event and clk = '1') then
```

rising_edge(clk)-上升沿
falling_edge(clk)-下降沿

```
            if enable = '0' then
```

```
                q_next <= d;
```

同步功能

```
            else
```

```
                null;
```

```
            end if;
```

```
        end if;
```

```
    end process;
```

```
    q <= q_next;
```

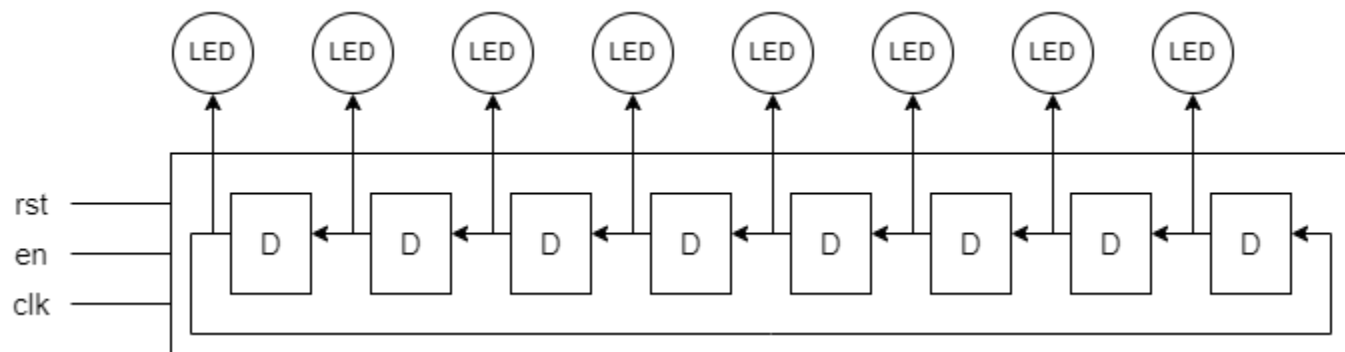
```
    q_n <= not q_next;
```

避免多余的寄存器引入，输出的
赋值放在process语句的外面

```
end Behavioral;
```

Running LED

- 流水灯：8个灯依次变暗



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity running_led is
    Port ( clk : in std_logic;
          rst : in std_logic;
          en_n : in std_logic;
          leds : out std_logic_vector(7 downto 0));
end running_led;
```

```
architecture Behavioral of running_led is
    signal leds_next : std_logic_vector(7 downto 0) := "11111110";
begin
    process(clk, rst)
    begin
        if (rst = '1') then
            leds_next <= "11111110";
        elsif (rising_edge(clk)) then
            if (en_n = '0') then
                leds_next <= leds_next(6 downto 0) & leds_next(7);
            else
                leds_next <= leds_next;
            end if;
        end if;
    end process;
    leds <= leds_next;
end Behavioral;
```

Running LED

- Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity running_led_tb is

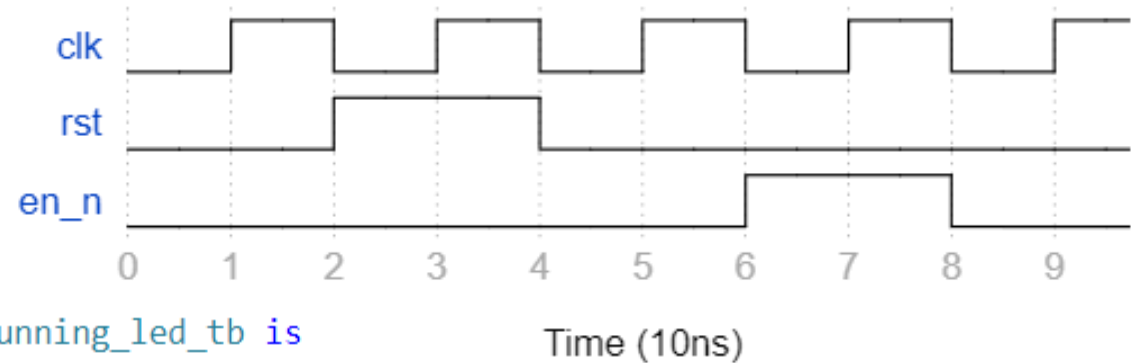
end running_led_tb;

end running_led_tb;
```

```
architecture tb of running_led_tb is
    component running_led is
        Port ( clk : in std_logic;
               rst : in std_logic;
               en_n : in std_logic;
               leds : out std_logic_vector(7 downto 0));
    end component;

    signal clk : std_logic := '0';
    signal rst : std_logic := '0';
    signal en_n : std_logic := '0';
    signal leds : std_logic_vector(7 downto 0);
    constant halfperiod : time := 10ns;

begin
    uut: running_led port map (clk=>clk, rst=>rst, en_n=>en_n, leds=>leds);
    clk <= not clk after halfperiod;
    rst <= '1' after halfperiod*2, '0' after halfperiod*4;
    en_n <= '1' after halfperiod*6, '0' after halfperiod*8;
end tb;
```



Running LED

- Constraints

```
## Clock signal
set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { clk }]; #I0_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}];
##Switches

set_property -dict { PACKAGE_PIN J15      IOSTANDARD LVCMOS33 } [get_ports { rst }]; #I0_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16      IOSTANDARD LVCMOS33 } [get_ports { en_n }]; #I0_L3N_T0_DQS_EMCCCLK_14 Sch=sw[1]

## LEDs

set_property -dict { PACKAGE_PIN H17      IOSTANDARD LVCMOS33 } [get_ports { leds[0] }]; #I0_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15      IOSTANDARD LVCMOS33 } [get_ports { leds[1] }]; #I0_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13      IOSTANDARD LVCMOS33 } [get_ports { leds[2] }]; #I0_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14      IOSTANDARD LVCMOS33 } [get_ports { leds[3] }]; #I0_L8P_T1_D11_14 Sch=led[3]
set_property -dict { PACKAGE_PIN R18      IOSTANDARD LVCMOS33 } [get_ports { leds[4] }]; #I0_L7P_T1_D09_14 Sch=led[4]
set_property -dict { PACKAGE_PIN V17      IOSTANDARD LVCMOS33 } [get_ports { leds[5] }]; #I0_L18N_T2_A11_D27_14 Sch=led[5]
set_property -dict { PACKAGE_PIN U17      IOSTANDARD LVCMOS33 } [get_ports { leds[6] }]; #I0_L17P_T2_A14_D30_14 Sch=led[6]
set_property -dict { PACKAGE_PIN U16      IOSTANDARD LVCMOS33 } [get_ports { leds[7] }]; #I0_L18P_T2_A12_D28_14 Sch=led[7]
```

时钟周期(ns) 时钟上升沿、下降沿时刻 时钟信号名 时钟引脚

创建时钟 时钟名称 时钟声明

设置属性 FPGA管脚 电平标准 顶层信号名

Running LED

- 板载时钟为100MHz，由于视觉暂留效应，8个灯看到都是常亮状态
- 目标：每1s切换一次
- 内置一个计数器，当计数到第100,000,000次时，LED灯切换一次并将计数器清零
- $\log_2 100,000,000 = 26.58$ ，因此需要27位二进制表示100,000,000
- 不建议用分频时钟作为clk信号
- 可以用Clock Wizard IP Core，但板载FPGA只支持4.687MHz~800MHz

Running LED

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;

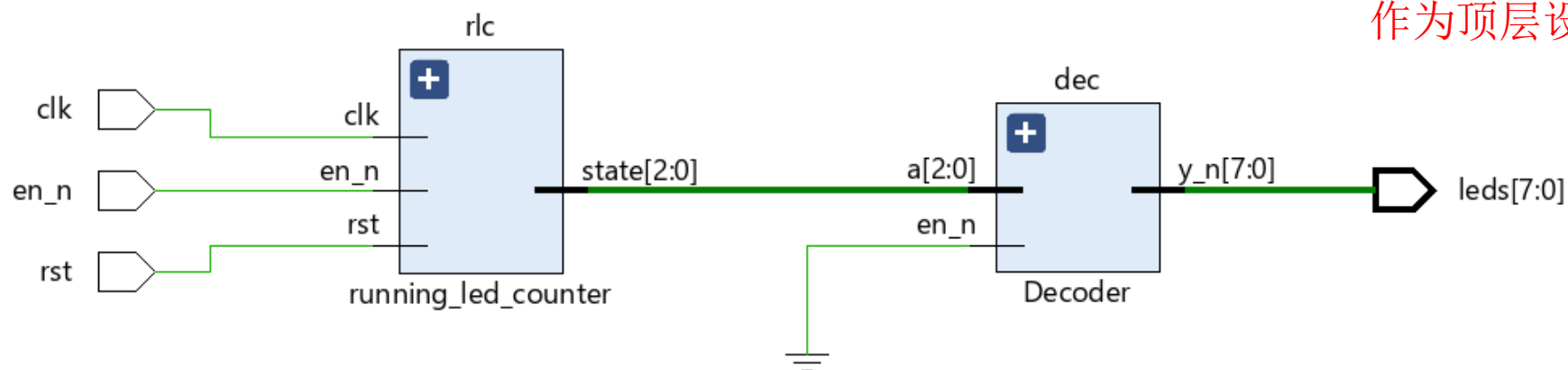
entity running_led_2 is
    Port ( clk : in std_logic;
           rst : in std_logic;
           en_n : in std_logic;
           leds : out std_logic_vector(7 downto 0));
end running_led_2;
```

```
architecture Behavioral of running_led_2 is
    signal leds_next : std_logic_vector(7 downto 0) := "11111110";
    constant max_count : integer := 99_999_999;
    signal counter : unsigned(26 downto 0) := (others => '0');
begin
    process(clk, rst)
    begin
        if (rst = '1') then
            leds_next <= "11111110";
        elsif (rising_edge(clk)) then
            if (en_n = '0') then
                if (counter = max_count) then
                    counter <= (others => '0');
                    leds_next <= leds_next(6 downto 0) & leds_next(7);
                else
                    counter <= counter + 1;
                end if;
            else
                leds_next <= leds_next;
            end if;
        end if;
        leds <= leds_next;
    end process;
end Behavioral;
```

Running LED

- 第二种思路

- 第一个模块设计从0到7的计数器（每秒计数1次）
- 第二个模块设计3-8译码器



set as top
可以指定文件
作为顶层设计

设计文件右键

Source Node Properties...	Ctrl+E
Open File	Alt+O
Replace File...	
Copy File Into Project	
Copy All Files Into Project	Alt+I
Remove File from Project...	Delete
Enable File	Alt+Equals
Disable File	Alt+Minus
Move to Simulation Sources	
Move to Design Sources	
Hierarchy Update	
Refresh Hierarchy	
IP Hierarchy	
Set as Top	
Set Global Include	
Clear Global Include	
Set as Out-of-Context for Synthesis...	
Set Library...	Alt+L
Set File Type...	
Set Used In...	
Edit Constraints Sets...	
Edit Simulation Sets...	
Add Sources...	Alt+A
Go to Source	F7

Running LED

- 第三种思路

- 从0到7的计数器即为一个有限状态机

```
architecture Behavioral of running_led_4 is
    signal state : unsigned(2 downto 0) := "000";
    signal state_next : unsigned(2 downto 0) := "000";
    constant max_count : integer := 99_999_999;
    signal counter : unsigned(26 downto 0) := (others => '0');
begin
```

```
    process(clk, rst)
    begin
        if (rst = '1') then
            state <= "000";
        elsif (rising_edge(clk)) then
            if (en_n = '0') then
                if (counter = max_count) then
                    counter <= (others => '0');
                    state <= state_next;
                else
                    counter <= counter + 1;
                end if;
            else
                state <= state;
            end if;
        end if;
    end process;
```

第一段
描述状态转移

时序
逻辑
电路

```
state_next <= state + 1;
```

```
process(state)
begin
    case state is
        when "000" =>
            leds <= "11111110";
        when "001" =>
            leds <= "11111101";
        when "010" =>
            leds <= "11111011";
        when "011" =>
            leds <= "11110111";
        when "100" =>
            leds <= "11101111";
        when "101" =>
            leds <= "11011111";
        when "110" =>
            leds <= "10111111";
        when "111" =>
            leds <= "01111111";
        when others =>
            leds <= "xxxxxxxx";
    end case;
end process;
```

第二段
判断下一状态

第三段描述
不同状态下的输出

组合
逻辑
电路

Assignment 1

以下设计为3位十进制计数器，请在Vivado中实现如下设计

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity decimal_counter is
port (CLK, RESET: in std_logic;
      d1, d10, d100: out std_logic_vector(3 downto 0));
end entity decimal_counter;

architecture concurrent_arch of decimal_counter is
signal d1_reg, d10_reg, d100_reg: std_logic_vector (3 downto 0);
signal d1_next, d10_next, d100_next: std_logic_vector (3 downto 0);
```

```
begin
-- register
process (CLK, RESET) is
begin
if RESET = '1' then
    d1_reg <= "0000";
    d10_reg <= "0000";
    d100_reg <= "0000";
elsif CLK'event and CLK='1' then
    d1_reg <= d1_next;
    d10_reg <= d10_next;
    d100_reg <= d100_next;
end if;
end process;
```

Assignment 1

```
-- next-state logic
d1_next <= "0000" when d1_reg = 9 else d1_reg+1;
d10_next <= "0000" when (d1_reg = 9 and d10_reg = 9) else
d10_reg+1 when d1_reg = 9 else d10_reg;
d100_next <= "0000" when (d1_reg=9 and d10_reg=9 and d100_reg=9) else
d100_reg+1 when (d1_reg=9 and d10_reg=9) else d100_reg;

-- Output logic
d1 <= d1_reg; d10 <= d10_reg; d100 <= d100_reg;

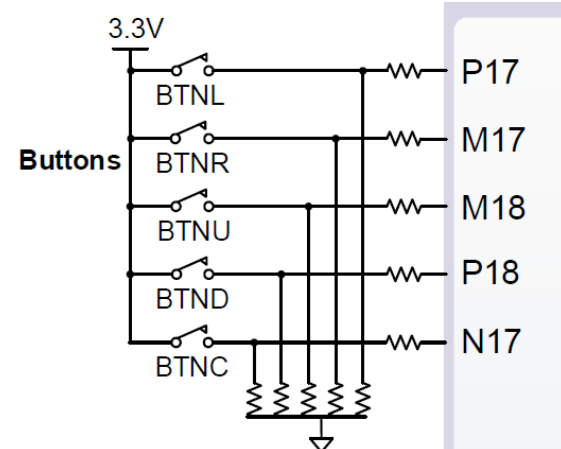
end architecture concurrent_arch;
```

Assignment 1

请修改以上程序，并在开发板上实现以下功能：

1. 将程序修改为每1s计数1次；
2. 为程序添加异步复位(reset)功能。复位功能使用LED右上方的任一按键。复位按键按下时进行复位，将计数归0；
3. 为程序添加同步置数(load)功能。置数功能使用最左手边的滑动开关，用右手边共12个开关表示3位数字的BCD码(每4个表示1位十进制数)。置数开关向上时，从12个开关读入数字，置数功能取消后计数器将从读入的数字开始继续计数；
4. 用最右手边的12个LED显示计数结果。

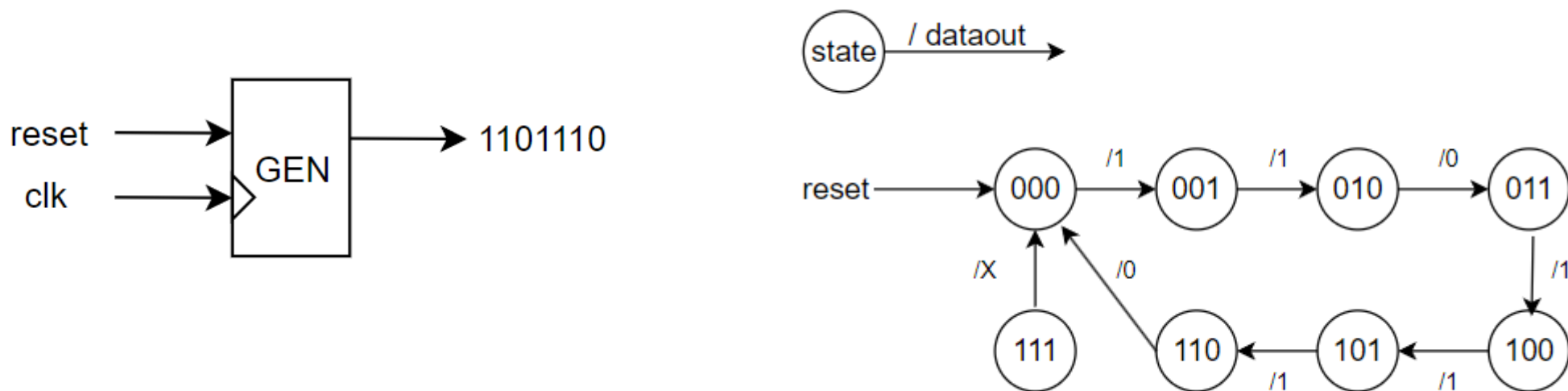
本题需要验收，请演示以上所有功能。



Assignment 2

- 序列信号发生器

按需要产生一组特定串行序列的电路被称为序列信号发生器。下图为“1101110”序列生成器的模块框图和状态转移图。如图所示，发生器有2个输入：第一个是复位信号，用于将状态初始化为000；第二个是时钟输入。



Assignment 2

1. 请编写VHDL程序满足以上设计要求;
2. 请编写testbench文件对设计进行仿真;
3. 请展示post implementation simulation results。

Assignment 3

- 101序列检测器

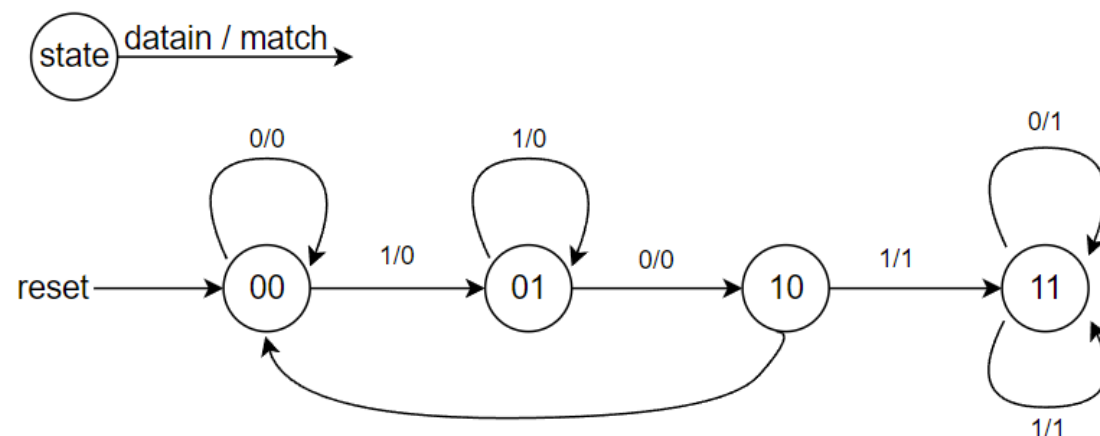
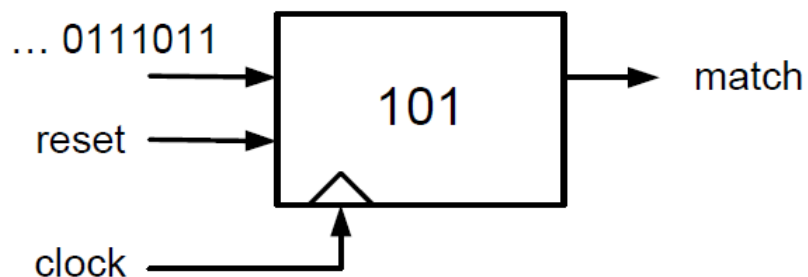
以下为序列检测器的模块框图和状态转移图。如图所示，检测器有3个输入：第一个是复位信号，用于将状态初始化为00；

第二个是位串行输入；

第三个是时钟输入。

检测器能够识别输入序列中的 101 序列，并将输出值设置为 1。

输出值将始终保持为 1，直到被复位。



Assignment 3

1. 请编写VHDL程序满足以上设计要求；
2. 请编写testbench文件对设计进行仿真；
3. 请展示post implementation simulation results。

题目2和3合并提交完整报告，包括：题目、原理、流程图、VHDL关键代码和testbench关键代码（合理注释）、结果（schematic、仿真波形）、描述并分析、结论